

مبانی و الگوریتم‌های کلان‌داده

جزوه جامع درسی و مرجع دانشگاهی تحلیل کلان‌داده

بر اساس سرفصل‌ها و اسلایدهای کتاب (MMDS) Datasets Massive of Mining
تألیف: ژوره لسکوک، آناند راجارامان، جفری اولمن

گردآورنده: احسان شهبازی

مجموعه یادداشتهای آموزشی کارشناسی ارشد

۷ مهر ۱۴۰۵

فصل ۲

نگاشت-کاهش و پشته نرم‌افزاری پردازش توزیع‌شده

۱.۲ مقدمه: چالش‌های معماری در مقیاس وب

در سیستم‌های سنتی، برای پاسخ‌گویی به افزایش حجم محاسبات، راهکار ارتقای عمودی^۱ (افزودن پردازنده، حافظه یا دیسک‌های سریع‌تر به یک سرور واحد) به کار گرفته می‌شد. با این حال، در عصر کلان‌داده، حجم داده‌ها به ده‌ها و صدها ترابایت رسیده است؛ در نتیجه ارتقای عمودی به دلیل محدودیت‌های فیزیکی، گذرگاه حافظه و هزینه‌های نجومی توجیه‌پذیر نیست.

رویکرد نوین، **ارتقای افقی**^۲ است؛ یعنی اتصال هزاران رایانه معمولی و ارزان‌قیمت تجاری^۳ در قالب خوشه‌های محاسباتی^۴. در این معماری:

- ماشین‌ها در قفسه‌هایی^۵ شامل ۳۰ تا ۴۰ گره مستقر شده و با سوئیچ‌های محلی به یکدیگر و با سوئیچ‌های شبکه اصلی به سایر قفسه‌ها متصل می‌شوند.

- نرخ خرابی قطعات سخت‌افزاری (خرابی هارد دیسک، سوختن منبع تغذیه، یا قطع کابل شبکه) بسیار بالا است. اگر میانگین طول عمر یک ماشین ۳ سال (۱۰۰۰ روز) باشد، در یک خوشه با ۱۰۰۰ ماشین، به طور متوسط هر روز حداقل یک ماشین دچار خرابی کامل می‌شود.

بنابراین، دو اصل بنیادین در معماری نرم‌افزارهای کلان‌داده حاکم است:

۱. **تحمل خطای نرم‌افزاری**^۶: سیستم نرم‌افزاری باید به گونه‌ای طراحی شود که خرابی گره‌ها را به عنوان امری روزمره بپذیرد و بدون نیاز به آغاز مجدد کل محاسبات، وظایف گره خراب را به طور خودکار بازتولید کند.

۲. **نزدیک‌سازی محاسبات به داده**^۷: انتقال ترابایت‌ها داده در پهنای باند محدود شبکه خوشه غیرممکن است؛ بنابراین کد برنامه باید به گره‌هایی که قطعات داده روی دیسک آن‌ها ذخیره شده است فرستاده شود.

۲.۲ سیستم‌های فایل توزیع‌شده (DFS / GFS / HDFS)

سیستم‌های فایل توزیع‌شده نظیر GFS^۸ و پیاده‌سازی متن‌باز آن HDFS^۹، زیرساخت بنیادین ذخیره‌سازی داده‌های حجیم به صورت پایدار و توزیع‌شده هستند.

Scale-Up یا Scaling Vertical^۱
Scale-Out یا Scaling Horizontal^۲
Commodity Hardware^۳
Computing Clusters^۴
Racks^۵
Software-level Fault Tolerance^۶
Moving Computation to Data^۷
Google File System^۸
Hadoop Distributed File System^۹

تعریف ۱.۲: تکه یا بلوک داده

در سیستم فایل توزیع‌شده، فایل‌های بسیار بزرگ به قطعات یکنواختی به نام **تکه^a** با اندازه‌ای بین ۶۴ تا ۱۲۸ مگابایت تقسیم می‌شوند. هر تکه به طور پیش‌فرض بر روی **سه گره متمایز** (ترجیحاً در دو قفسه شبکه مختلف جهت تاب‌آوری در برابر قطع برق قفسه) تکثیر^b می‌گردد.

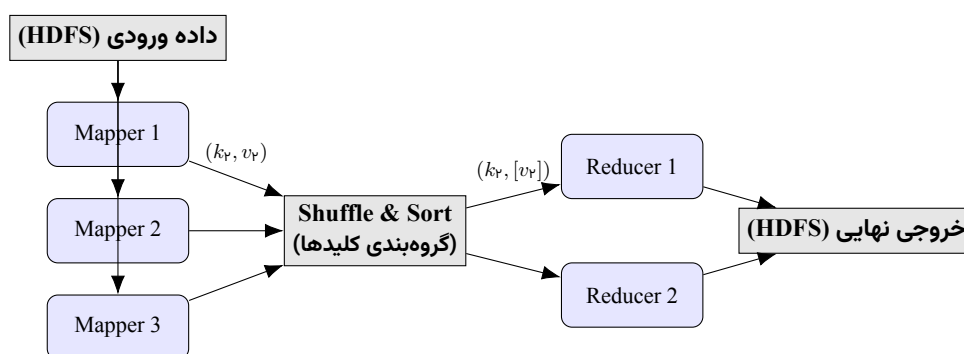
Block یا Chunk^a
Replication^b

معماری HDFS از دو بخش اصلی تشکیل شده است:

- **گره نام (NameNode):** متادیتا و شاخص محل ذخیره‌سازی تکه‌ها را در حافظه اصلی خود نگه می‌دارد. جهت کارایی بالا، هیچ داده خامی از مسیر گره نام عبور نمی‌کند.
- **گره‌های داده (DataNodes):** مسئولیت ذخیره‌سازی واقعی تکه‌های فایل روی دیسک محلی و ارائه مستقیم آن‌ها به کلاینت‌ها را بر عهده دارند.

۳.۲ مدل برنامه‌نویسی نگاشت-کاهش (MapReduce)

مدل نگاشت-کاهش توسط جفری دین و سانجای جماوات در گوگل معرفی شد [۶]. این مدل، پیچیدگی‌های موازی‌سازی، زمان‌بندی، تقسیم بار شبکه و مدیریت خرابی را از دید برنامه‌نویس پنهان ساخته و محاسبات را به دو تابع اصلی تقسیم می‌کند:



شکل ۱.۲: جریان اجرای الگوریتم MapReduce شامل مراحل نگاشت، بُر زدن و مرتب‌سازی، و کاهش

تعریف ۲.۲: امضای توابع نگاشت و کاهش

توابع Map و Reduce به شکل زیر فرمول‌بندی می‌شوند:

$$\text{Map}(k_1, v_1) \rightarrow \text{list}(k_2, v_2) \quad (1.2)$$

$$\text{Reduce}(k_2, \text{list}(v_2)) \rightarrow \text{list}(v_3) \quad (2.2)$$

در مرحله میانی که توسط زیرساخت اجرا می‌شود و به **بُر زدن و مرتب‌سازی^a** مشهور است، تمامی مقادیر با کلید یکسان k_2 گردآوری شده و به صورت یک لیست تحویل یک کاهنده واحد می‌گردند.

Shuffle and Sort^a

مثال کاربردی ۱.۲: شمارش واژگان (Word Count)

مسئله استاندارد برای نشان دادن کارکرد MapReduce، شمارش تکرار هر کلمه در یک مجموعه سند است:

- **ورودی نگاشت:** کلید نام سند و مقدار متن یک سطر از سند است.

• تابع نگاشت:

(۳.۲) منتشر شود. $(w, 1)$ در متن، جفت w برای هر واژه $\text{Map}(\text{DocID}, \text{Text}) \Rightarrow$

• مرحله میانی (سیستم): تمام جفت‌های واژه یکسان دسته‌بندی می‌شوند:

(۴.۲) $(w, [1, 1, 1, \dots, 1])$

• تابع کاهش: مقادیر لیست را جمع می‌زند:

(۵.۲) $\text{Reduce}(w, [v_1, v_2, \dots, v_n]) \Rightarrow \left(w, \sum_{i=1}^n v_i \right)$

۴.۲ بهینه‌سازی‌های اجرایی در نگاشت-کاهش

۱.۴.۲ تابع ترکیب‌کننده (Combiner)

در مثال شمارش واژگان، برای کلماتی مانند «در» یا «از»، یک مپر ممکن است میلیون‌ها جفت $(1, \text{«در»})$ تولید کند. فرستادن این حجم از داده خام روی شبکه باعث انسداد پهنای باند می‌گردد.

تعریف ۳.۲: ترکیب‌کننده

تابع ترکیب‌کننده^a یک مینی-ردیوسر اختیاری است که در حافظه ماشین نگاشت، قبل از ارسال داده‌ها به شبکه اجرا می‌شود. شرط استفاده از Combiner این است که عمل کاهش دارای ویژگی شرکت‌پذیری^b و جابجایی‌پذیری^c باشد (مانند جمع، ضرب، یا محاسبه بیشینه و کمینه؛ اما برای محاسبه میانگین نمی‌توان مستقیماً میانگین گرفت، بلکه باید مجموع و تعداد به طور مجزا فشرده شوند).

Combiner^a
Associative^b
Commutative^c

۲.۴.۲ تابع افرازبندی (Partitioner)

برای تعیین اینکه هر کلید k_p به کدام کاهنده (از میان R پردازنده کاهنده) ارسال شود، از تابع افرازبندی استفاده می‌شود. به صورت پیش‌فرض:

(۶.۲) $\text{Reducer_ID} = \text{hash}(k_p) \pmod{R}$

با تنظیم دستی تابع افرازبندی می‌توان توزیع کلیدها را برای جلوگیری از کجی داده^{۱۰} کنترل کرد.

۳.۴.۲ مدیریت گره‌های کند (Stragglers) با اجرای گمانه‌زانه

در یک اجرای توزیع‌شده با هزاران وظیفه، اگر ۹۹٪ وظایف ظرف ۵ دقیقه تمام شوند، ولی یک وظیفه روی ماشینی با هارد دیسک معیوب گیر کند، کل جاب متوقف می‌ماند. این پدیده عقب‌ماندگی یا گره کند^{۱۱} نام دارد. راهکار سیستم MapReduce اجرای کارهای پشتیبان^{۱۲} در مراحل پایانی است؛ هر یک از کارهای موازی که زودتر به پایان برسد، پذیرفته شده و دیگری متوقف می‌گردد.

Data Skew^{۱۰}
Straggler^{۱۱}
Execution Speculative یا Tasks Backup^{۱۲}

۵.۲ پیاده‌سازی اعمال جبر رابطه‌ای در MapReduce

پایگاه داده‌های رابطه‌ای بر مبنای اعمال جبر رابطه‌ای عمل می‌کنند. این اعمال در MapReduce به صورت مقیاس‌پذیر پیاده‌سازی می‌شوند:

۱.۵.۲ پیوند طبیعی (Natural Join)

فرض کنید دو رابطه $R(A, B)$ و $S(B, C)$ داریم و می‌خواهیم $R \bowtie S$ را روی ویژگی مشترک B محاسبه نماییم.

الگوریتم ۱.۲: پیوند طبیعی در MapReduce

• ورودی: سطرهای رابطه R به فرم (a, b) و سطرهای رابطه S به فرم (b, c) .

• خروجی: تاپل‌های پیوندخورده (a, b, c) .

• تابع نگاشت:

$$(7.2) \quad (b, (R, a)) \Rightarrow \text{خروجی: } R(a, b) \text{ برای هر سطر}$$

$$(8.2) \quad (b, (S, c)) \Rightarrow \text{خروجی: } S(b, c) \text{ برای هر سطر}$$

• تابع کاهش: ورودی کاهنده برای کلید b شامل تمام مقادیر مرتبط با b از هر دو رابطه است:

$$(9.2) \quad (b, [(R, a_1), (R, a_2), \dots, (S, c_1), (S, c_2), \dots])$$

کاهنده لیست را به دو زیرمجموعه بر حسب برچسب R و S تقسیم کرده و حاصل ضرب دکارتی آن‌ها را تولید می‌کند: به ازای هر $a \in R$ و هر $c \in S$ ، تاپل (a, b, c) را منتشر می‌نماید.

۲.۵.۲ گروه‌بندی و تجمیع (Grouping and Aggregation)

فرض کنید رابطه $R(A, B, C)$ را داریم و می‌خواهیم عملگر $\gamma_{A, \text{SUM}(B)}(R)$ (معادل دستور SELECT A, SUM(B) FROM R GROUP BY A) را در نگاشت-کاهش پیاده کنیم:

• تابع نگاشت: برای هر تاپل (a, b, c) ، خروجی جفت (a, b) خواهد بود.

• تابع ترکیب‌کننده (Combiner): در حافظه ماشین می‌رود، مجموع مقادیر b به ازای هر a محاسبه و جمع محلی ارسال می‌شود.

• تابع کاهش: به ازای هر کلید a ، لیست مقادیر $[b_1, b_2, \dots]$ جمع زده شده و جفت $(a, \sum b_i)$ منتشر می‌گردد.

۳.۵.۲ ضرب ماتریس در بردار (Matrix-Vector Multiplication)

مسئله ضرب ماتریس بزرگ M با ابعاد $n \times n$ در بردار v به طول n (که در فصل پنجم برای الگوریتم PageRank حیاتی است):

$$(10.2) \quad x_i = \sum_{j=1}^n m_{ij} v_j$$

حالت اول: بردار v در حافظه اصلی هر می‌رود:

• هر می‌رود یک سطر یا مجموعه‌ای از درایه‌های ماتریس m_{ij} را می‌خواند.

• می‌رود بلافاصله حاصل ضرب $m_{ij} v_j$ را محاسبه کرده و جفت $(i, m_{ij} v_j)$ را به عنوان خروجی صادر می‌کند.

• کاهنده تمام مقادیر با کلید i را جمع می‌زند تا درایه نهایی x_i محاسبه شود.

حالت دوم: بردار v در حافظه جا نمی‌شود (حجم بسیار بالا): ماتریس M و بردار v به بلوک‌های مربعی و نواری تقسیم می‌شوند. در این حالت، نگاشت‌ها بر اساس شناسه بلوک‌ها سازمان‌دهی شده و محاسبات به صورت بلوکی صورت می‌پذیرد.

۴.۵.۲ ضرب دو ماتریس بزرگ در نگاشت-کاهش (Matrix Multiplication)

فرض کنید هدف محاسبه حاصل ضرب دو ماتریس بزرگ M با ابعاد $r \times c$ و N با ابعاد $c \times p$ باشد $P = M \times N$ به طوری که $p_{ij} = \sum_{k=1}^c m_{ik} n_{kj}$. دو الگوریتم استاندارد در مرجع MMDS برای این منظور تدوین شده است: ۱. الگوریتم دو مرحله‌ای نگاشت-کاهش (Two-Pass MapReduce):

۱. گذر اول (محاسبه ضرب‌های جزئی):

- نگاشت: برای هر درایه m_{ik} جفت $(k, (M, i, m_{ik}))$ و برای هر درایه n_{kj} جفت $(k, (N, j, n_{kj}))$ تولید می‌شود.

- کاهش: کاهنده برای کلید مشترک k ، تمام درایه‌های سطر k از N و ستون k از M را در اختیار دارد و به ازای هر جفت، ضرب جزئی $m_{ik} n_{kj}$ را با کلید خروجی (i, j) صادر می‌کند:

$$((i, j), m_{ik} n_{kj}) \quad (11.2)$$

۲. گذر دوم (مجموع ضرب‌های جزئی):

- نگاشت: تابع همانی (خروجی گذر اول را مستقیماً عبور می‌دهد).

- کاهش: به ازای هر کلید (i, j) ، تمام ضرب‌های جزئی دریافتی را با یکدیگر جمع کرده و درایه نهایی $p_{ij} = \sum_{k=1}^c m_{ik} n_{kj}$ را در فایل خروجی ذخیره می‌کند.

۲. الگوریتم تک‌مرحله‌ای (One-Pass MapReduce): اگر اولویت کاهش تعداد گذرها و زمان پایان باشد:

- هر درایه m_{ik} توسط میر به p نسخه تکثیر شده و با کلیدهای $((i, 1), (M, k, m_{ik}))$ تا $((i, p), (M, k, m_{ik}))$ صادر می‌گردد.

- هر درایه n_{kj} نیز به r نسخه با کلیدهای $((1, j), (N, k, n_{kj}))$ تا $((r, j), (N, k, n_{kj}))$ تکثیر می‌شود.

- ردیوسر متناظر با کلید (i, j) ، کل سطر i از M و کل ستون j از N را به یکباره دریافت کرده و p_{ij} را مستقیماً در یک گام محاسبه می‌نماید (این روش ترافیک میر به شبکه را به اندازه ابعاد ضرب افزایش می‌دهد).

نکته امتحانی و کلیدی

در تحلیل هزینه الگوریتم‌های MapReduce در آزمون، دو معیار اساسی وجود دارد: ۱. **هزینه ارتباطات (Communication Cost)**: برابر است با مجموع اندازه فایل‌های خروجی تمام میپرها که باید از طریق شبکه به سمت ردیوسرها منتقل شوند. ۲. **هزینه زمانی سپری‌شده (Elapsed Time)**: بیشترین زمان صرف‌شده در طولانی‌ترین مسیر اجرای یک گره، که نشان‌دهنده گلوگاه اصلی محاسبات موازی است.

جمع‌بندی فصل دوم

- مدل نگاشت-کاهش پردازش داده‌های در مقیاس پتابایت را با تکیه بر سخت‌افزارهای تجاری ارزان و تحمل خطای خودکار میسر می‌سازد.
- سیستم‌های فایل توزیع‌شده (HDFS) با تکثیر بلوک‌ها تضمین پایداری داده را بر عهده دارند.
- توابع Combiner و Partitioner نقش حیاتی در بهینه‌سازی پهنای باند و تعادل بار شبکه ایفا می‌کنند.
- پیوند طبیعی، گروه‌بندی و ضرب‌های ماتریسی به سادگی و کارایی در قالب یک یا چند گام نگاشت-کاهش مدل‌سازی می‌شوند.